

Architectural Coverage Framework Using Reference Model

Sharvil Panandikar, Darshak Koshiya, Jiahan
Zhang
Tenstorrent

The logo for 'snug' is displayed within a white circle. The text 'snug' is in a lowercase, sans-serif font, with 'sn' in purple and 'ug' in orange. The background of the slide features a teal color with a network of white lines and dots, and a large purple circle on the right side.

Agenda

Agenda

- Motivation
- Overview
- Coverage Generation
- Coverage Sampling
- Results
- Limitations

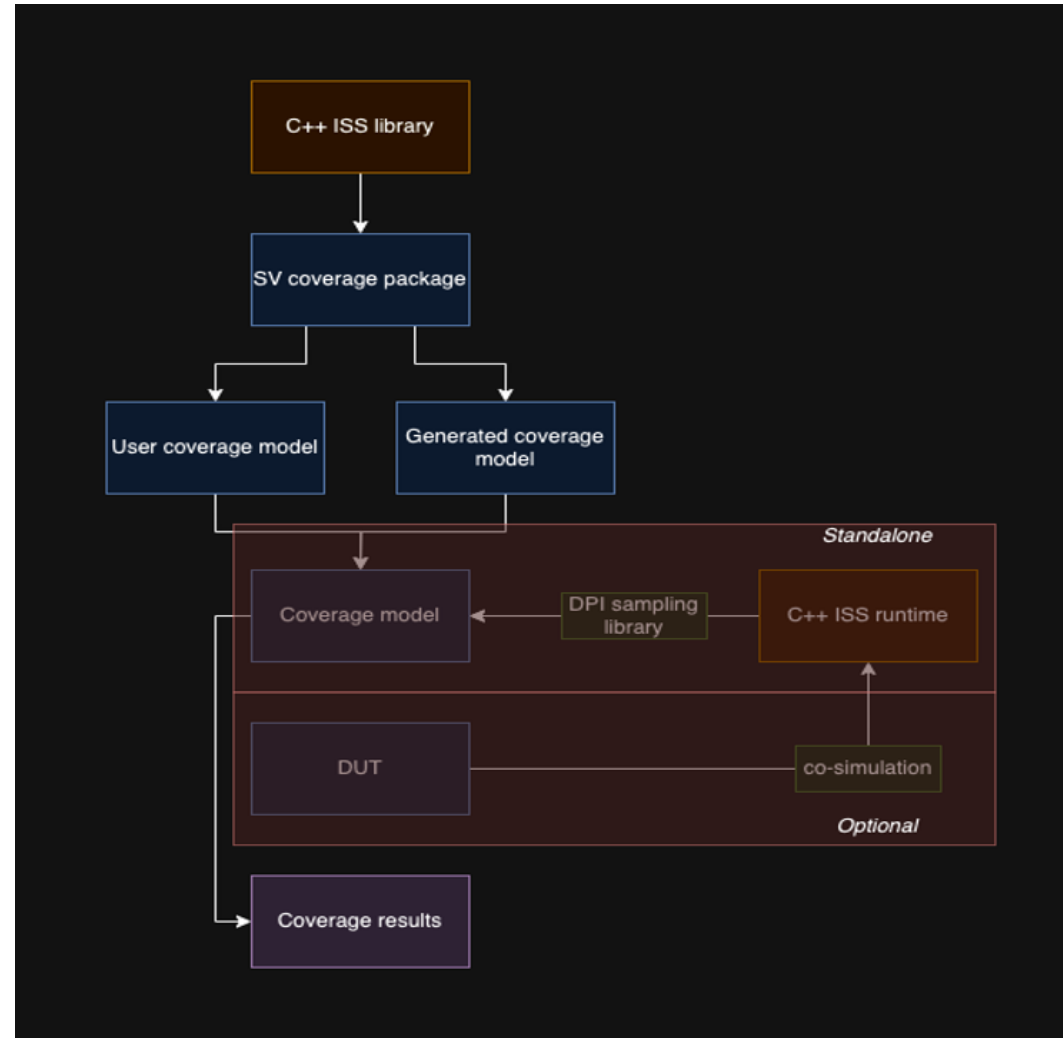
Motivation

- Coverage is indispensable with growing complexity of testbenches
 - With each feature added, the scope of the coverage model increases drastically
 - Hand-written coverage is no longer an option as its error-prone and tiring over large coverage space
 - Most proprietary coverage flows limit the use of SystemVerilog language features
- Early feedback on coverage can help to identify stimulus holes
 - During initial phases of test planning, its helpful to have a quick coverage feedback to determine quality of stimulus
- Reuse of reference model for coverage model generation
 - Reference models are an integral part of a Testbench for providing golden results
 - This makes the reference model an implementation of the specification
 - We can use this aspect of the reference model to generate coverage model

Overview

Overview

10K feet view of the workflow



Overview

- The coverage flow relies on two parts :
- Coverage Model Generation
 - In this step, we query the ISS for coverage specific information and generate a SystemVerilog package
 - The package includes autogenerated coverage as well as artifacts for users to write coverage
- Coverage Sampling
 - This is a runtime process where we run stimulus, either in ISS standalone mode or RTL cosimulation
 - The ISS samples and sends the values over DPI to a SystemVerilog layer that records the coverage

Coverage Generation

Coverage Generation

- The coverage generation is based on :
 - ISS library
 - The ISS library is a self-contained library, which queries the ISS to capture potentially relevant coverage information
 - The information is then translated to generate SystemVerilog package
 - SystemVerilog Package
 - Inside the package, we represent the information as
 - Enumerations
 - Classes
 - Global variables
 - The package can then be imported by user-defined coverage model to write coverage for specific cases
 - This allows use to benefit from the autogenerated coverage while using SystemVerilog Language features

Coverage Generation

- Sample ISS API for page sizes
 - The ISS stores the information for all the available page sizes, given the paging mode.
 - When queried by the coverage generator, the ISS library provides the relevant information as a C struct.
 - The generator then translates the C struct to SystemVerilog equivalent datatype.
 - Since there are finite number of page sizes, a SystemVerilog enum is generated

```
static constexpr const char* ptePageSize(Mode m)
{
    if (m == Mode::Bare)
        return "";

    ...

    if (m == Mode::Sv32)
    {
        if (level == 1) return "4M";
    }
    else if (m == Mode::Sv39)
    {
        if (level == 1) return "2M";
        if (level == 2) return "1G";
    }

    ...
}
```

Coverage Generation

Inside the SystemVerilog package

```
typedef enum {  
    POINT_PRIVILEGEMODE=13,  
  
    POINT_NEXTPRIVILEGEMODE=14,  
  
    ....  
} archInfoPoints_e;
```

Autogenerated enumeration that captures all the available coverpoints

```
class satp_csr;  
  
    logic[63:0] value;  
    logic[43:0] ppn;  
    logic[15:0] asid;  
    logic[3:0] mode;  
  
    covergroup cg_satp with function sample (logic[15:0] asid, logic[3:0] mode, logic[43:0] ppn);  
        option.per_instance = 1;  
        option.name = "satp";  
        coverpoint asid iff (csrenum_var == CSRENUM_SATP)  
        {  
            bins zeroes = {0};  
            bins ones = {1};  
        }  
        coverpoint mode iff (csrenum_var == CSRENUM_SATP)  
        {  
            bins valid[] = {[0:$]};  
        }  
    endgroup  
  
endclass
```

Autogenerated SystemVerilog class for Control Register

Coverage Sampling

Coverage Sampling

- The coverage sampling is runtime feature that relies on :
 - ISS sampling support at specific points in the simulation
 - Receiving values collected at specific events and sampling autogenerated and user covergroups that are relevant to that event
- The sampled values from ISS are sent over DPI to the SystemVerilog side
- The SystemVerilog layer then filters the values according to needs of the coverage model
- Coverage is the recorded by calling sample() method for relevant covergroups
- The sampling library is linked to the ISS such that it does not have any RTL dependency
- This allows us to run the simulation in standalone ISS mode

Coverage Sampling

- ISS library implementation to record coverage values
 - While running the stimulus, the ISS creates a table, which is a data structure that holds a key-value pair, with the coverpoint as the key and its corresponding value. The generator then translates the C struct to SystemVerilog equivalent datatype.
 - In the example, for every load/store access, we record the virtual and physical addresses.

```
uint64_t virtLdStAddr, physLdStAddr = ~0;
uint64_t ldStSize = tr->lastLdStAddress(virtLdStAddr, physLdStAddr);
if (ldStSize)
{
    table.addEntry(Point::VirtLdStAddr, virtLdStAddr);
    table.addEntry(Point::PhysLdStAddr, physLdStAddr);
    ...
}
```

Coverage Sampling

```
export "DPI-C" function sample_sv;

Point p = static_cast<Point>(table.entry.first);
if (p == Point::EndOfSim) {
    // Exit sim
} else {
    sample_sv(&pkt);
}
```

Example of an SV layer
implementation for receiving values
from ISS over DPI

```
typedef struct {
    uint64_t cp;
    uint64_t val;
} cp_pkt;
```

C struct to send coverage values over DPI

Results

Results : Example

- We have demonstrated an example with some highlights on the implementation :
 - We use Whisper, which is an open-source RVA23 compliant ISS
 - We use Synopsys VCS for simulation framework along with urg utility to generate coverage report
 - For our case, we are using the privilege mode feature as defined in RISC-V privileged ISA
 - During the coverage generation phase, we query the C++ ISS library, which provides the relevant information for privilege mode parameters
 - This information is then used by the coverage generation framework to auto-generate a SystemVerilog equivalent enum
 - The autogenerated coverage module then creates covergroups for the enum parameter

```
typedef enum bit [63:0] {  
    PRIVILEGEMODE_MACHINE=64'd3,  
    PRIVILEGEMODE_RESERVED=64'd2,  
    PRIVILEGEMODE_SUPERVISOR=64'd1,  
    PRIVILEGEMODE_USER=64'd0  
} PrivilegeMode_e;  
PrivilegeMode_e privilegemode_var; |
```

```
covergroup cg_privilege_mode;  
    cp_privilege_mode: coverpoint privilegemode_var;  
endgroup
```

Results : Example

- The user has access to the above package and can reuse the `privilegemode_var` variable in their own covergroups to create interesting coverage scenarios.
- We can then invoke the sampling calls to the covergroup and pass the value that is received on an update to the privilege mode from the C++ ISS runtime library.
- When the instruction is retired or suffers a trap, all collected coverage information in the ISS is provided to the sampling library

```
privilegemode_var = PrivilegeMode_e'(<value coming from ISS runtime during sampling>);  
cg_privilege_mode.sample();
```

Excluded/Illegal bins

NAME	COUNT	STATUS
auto_PRIVILEGEMODE_RESERVED	0	Excluded
reserved_priv	0	Excluded

Covered bins

NAME	COUNT	AT LEAST	STATUS	TEST	COUNT
auto_PRIVILEGEMODE_MACHINE	367470927	1		T1	367470
auto_PRIVILEGEMODE_SUPERVISOR	107306351	1		T1	107306
auto_PRIVILEGEMODE_USER	63602252	1		T1	636022

Limitations

- Microarchitectural Coverage :
 - This flow currently supports only architectural coverage i.e coverage that can be inferred from ISA
 - The microarchitectural coverage is implementation specific and hence cannot be generalized
- Large width variables :
 - Due to the DPI interfaces during sampling, the simulations can slow down for large-width variables as seen with vectors, SIMD instruction operands etc
 - These cases also need considerable processing of the sampled values for coverage, for example, coverage involving large-width vectors would need a different DPI interface or require multiple DPI calls to be handled.

References

References

- Riscv/Riscv-Isa-Manual. 2017. RISC-V, 15 Jan. 2025. GitHub, <https://github.com/riscv/riscv-isa-manual>
- Tenstorrent/Whisper. 2021. Tenstorrent AI ULC, 14 Jan. 2025. GitHub, <https://github.com/tenstorrent/whisper>.
- Bazelbuild/Bazel. 2014. Bazel, 15 Jan. 2025. GitHub, <https://github.com/bazelbuild/bazel>
- Riscv/Riscv-Profiles. 2021. RISC-V, 26 Jan. 2025. GitHub, <https://github.com/riscv/riscv-profiles>

THANK YOU

Our
Technology,
Your
Innovation™